dareskro on Solana

The payroll vault nobody owns — the AI reads, the code computes, a human approves, the program pays.

Encode Solana Hackathon 2026 · Best Consumer App · Aurora Intents
Program on devnet `6NoVNiHvEsagkYYobHjrwQgkqL5xHp5zGStZK4YtdPSu` · github.com/LeDenti/dareskro · solana/

The problem: paid for work nobody can fully verify

A pharmacist opens the on-call pharmacy on a Sunday. The collective agreement pays those hours more. Who proves them?

Every month, someone chases a salary already earned. The one who pays holds both the money and the proof.

Same shape, other jobs

- a freelancer's delivery, judged against a spec
- a sales commission, judged against a contract
- a supplier's invoice, advanced before the debtor pays

What is missing

Not a promise. A place where the money sits that the payer can no longer empty or redirect, and a rule for who opens it.

The rule

The agent reads. The code computes. A human approves. Solana pays.

- **The AI agent** gathers the evidence: the rota, the badge log, the Sunday hours, the invoice dossier. It writes a memo. It never moves money.
- **The code** computes the amount line by line; anyone can replay it.
- **A human** approves the statement with a signature. Nothing moves before it.
- **The program** locks the money with one possible outcome, and on the date pays the one address set at creation. Not before. Not to anyone else.

How Solana powers it: a program with no owner

- Solana has no native escrow, so the Anchor program **is** the escrow: the funds sit in a PDA nobody has the key to, in SOL or USDC.
- **Time-locked** (payroll): no release before the date by the cluster clock, no cancel ever, not even by the employer; anyone may release after, only to the destination written at creation.
- **Hash-locked** (delivery): only the secret releases, and the secret becomes public in that same call.
- Several escrows in one transaction: all or nothing (net pay + contributions + tax).

Measured, not promised

- 15 tests against a local validator, one per guarantee
- every refusal replayed on devnet with its signature in the README: `TooEarly`, `NotCancellable`, `WrongDestination`…
- the sealed payslip: a non-transferable NFT, readable by the named readers only
- a shared vault, 2 of 3: the rental deposit, run with Phantom

A consumer app: one page, no terminal

- Phantom, or a throwaway devnet key created in the browser
- recipient, amount in dollars, date — lock
- release, cancel, close, each with its explorer link; the refusals are shown as the program returns them
- the payer chooses what to pay with: **SOL, XRP or NEAR** — Aurora quotes live
- the fee per act is shown before the lock, and paid on top: what the recipient gets is never touched

Live on devnet: `ledenti.github.io/dareskro/`

The pay desk, same desk, another ledger

- the dareskro pay desk (XRPL, NEAR, PayPal) has a Solana tab: the agent reads the proof, the code computes, the human approves, the desk locks on Solana
- the registry is the chain itself: `listEscrows({ owner })` is a `getProgramAccounts`

Aurora Intents: the employer pays from the chain they hold

37 s

XRP on the XRP Ledger → **3 USDC on Solana**, live on mainnet, 1 November 2026. Swap API: quote, deposit address, status polled to SUCCESS, refund path handled.

XRPL tx 56205926… · Solana tx 58Az4DDm…

- no bridge for the user to manage: one deposit address per quote, Aurora routes
- the page quotes XRP and NEAR for every lock; the desk records what was asked and what Aurora applied
- measured and written down: Aurora's own 10 bps fee is applied whatever `appFees` says — no integrator cut on this key
- next: Intents Connect, deposit and lock in one flow

The invoice advance: the program holds the ceilings

> On the XRP Ledger, two signers out of three passed an advance above the 30 % concentration cap: the ledger knew no cap. On Solana the program holds it.

step	what the program did on devnet, 5 November
pool	2 of 3 (AI seat, risk policy, operator), capital 160, caps written at creation; no proposal without the hash of the AI seat's memo
F-001	45 advanced on 50, AI seat + policy: paid, 2 of 3
F-002	54 on 160 = 33.75 %: AI seat + operator → `ConcentrationBreached`; the third signer changes nothing
F-003	36 advanced, 10 repaid, default after due + grace: the lenders carry the loss, nothing is seized; the supplier buys it back
lenders	A leaves before the buyback: 67.45 for 80; B leaves after: 94.45 for 80

Also: a fee per act bounded by the program (≤ 1 %, floor ≤ 0.01 SOL), zero by default, paid on top of the lock.

What works, what's next, what we say honestly

Works today, on devnet

- payroll and delivery escrows, SOL and USDC
- sealed payslip, shared vault 2/3, fee per act, invoice advance
- the page, the desk, Aurora on mainnet

Next

- Intents Connect · USDC in the pool · policy administrator under compiled caps · mainnet with the upgrade authority revoked

Honest limits

- devnet only; on mainnet "no owner" means the upgrade authority is burnt
- the AI seat reads off-chain; the program sees its signature and the hash of its memo
- the invoice advance is a lending activity: the licence question is open
- no French administration publishes a reading key yet

Where to look

code	github.com/LeDenti/dareskro/tree/main/solana — README with every measured run and its signatures
program	devnet `6NoVNiHvEsagkYYobHjrwQgkqL5xHp5zGStZK4YtdPSu`
try it	ledenti.github.io/dareskro/ — a burner key, a lock, a release, on devnet
film	youtube.com/watch?v=Uen4K_TKGLo — 3 minutes, the Sunday shift
release	github.com/LeDenti/dareskro/releases/tag/v0.3-solana-final — films, this deck
prior work	the XRPL version, XRPL Hackathon NYC, October 2026; NEAR and PayPal rails of the same desk

The agent reads. The code computes. A human approves.
Solana pays.